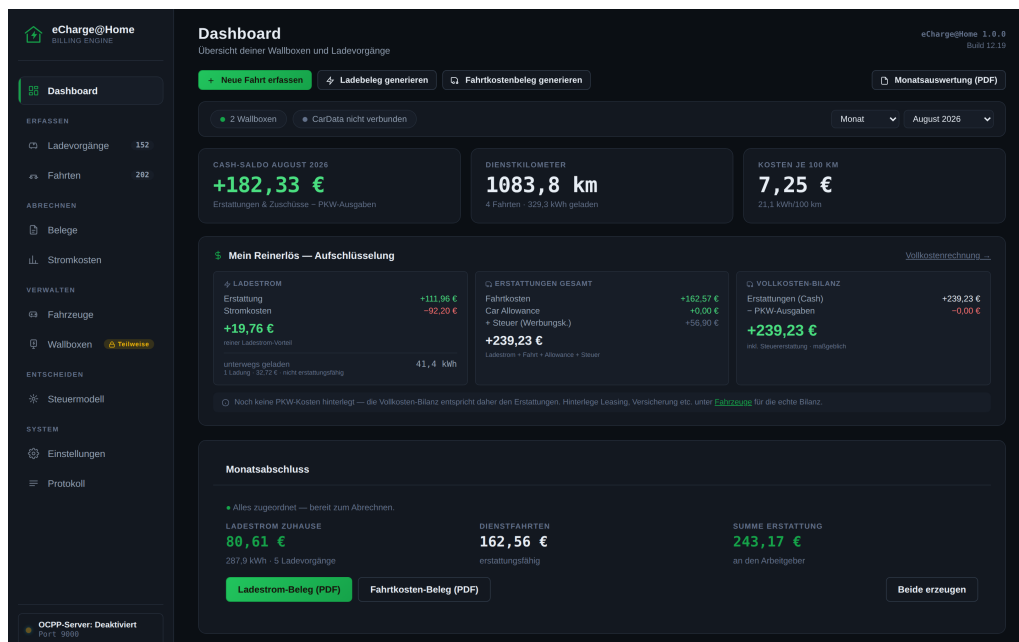


eCharge@Home

Installation

Windows · macOS · Linux · Docker · NAS



Fassung 1.0 · Stand August 2026

Der schnellste Weg

Wer Docker hat, ist in zwei Minuten fertig. Das fertige Abbild liegt öffentlich bereit — es muss nichts entpackt und nichts gebaut werden.

```
mkdir -p ~/echarge && cd ~/echarge
curl -O
https://raw.githubusercontent.com/STL2020/charge-at-home/main/docker-compose.ghcr.yml
docker compose -f docker-compose.ghcr.yml up -d
```

Aufrufen unter `http://<IP-des-Geräts>:8501`

Auf Windows und macOS mit Docker Desktop genügt sogar eine Zeile:

```
docker run -d -p 8501:8501 -v echarge-data:/srv/data
ghcr.io/stl2020/charge-at-home:latest
```

Die folgenden Seiten beschreiben die Wege im Einzelnen — für alle, die kein Docker haben oder die Einrichtung geführt haben möchten.

Welcher Weg passt?

Es gibt zwei Wege. Der Unterschied entscheidet darüber, ob nächtliche Ladevorgänge erfasst werden — deshalb steht er ganz vorn.

	Docker (NAS, Pi, Server)	Lokal (Windows, Mac, Linux)
Rechner muss laufen	nein — das Gerät läuft ohnehin	ja, dauerhaft
Nach Stromausfall	startet selbsttätig	von Hand
Wallbox über OCPP	uneingeschränkt	nur bei geöffnetem Programm
Einrichtung	ein Befehl mit dem fertigen Abbild	Doppelklick
Erster Start	ein bis zwei Minuten	einige Minuten
Zugriff vom Handy	ja, im Heimnetz	ja, solange der Rechner läuft

Kurz gesagt: Wer nur Fahrten und Ladungen erfassen und Belege erzeugen will, ist mit der lokalen Installation gut bedient. Wer eine Wallbox über OCPP anbinden möchte, sollte Docker auf einem durchlaufenden Gerät nutzen.

Voraussetzungen

Weg	Was benötigt wird
Docker	Ein Gerät mit Docker: Synology, QNAP, Unraid, Raspberry Pi 4 oder neuer, oder ein Rechner mit Docker Desktop
Windows	Windows 10 oder 11. Python 3.10 oder neuer, falls kein Docker installiert ist
macOS	macOS 12 oder neuer. Python 3.10 oder neuer
Linux	Beliebige Distribution mit Python 3.10 oder neuer

Windows

Für Windows liegt ein Installationsskript bei. Es prüft, was auf dem Rechner vorhanden ist, installiert Fehlendes nach und legt auf Wunsch eine Verknüpfung an.

Schritt 1 — Entpacken

1. Die heruntergeladene Datei `eCharge_v12.10.zip` im Explorer suchen — meist unter **Downloads**.
2. **Rechtsklick** → **Alle extrahieren ...**
3. Als Ziel einen dauerhaften Ort wählen, etwa `C:\eCharge` oder `Dokumente\eCharge`. **Nicht im Downloads-Ordner belassen** — dort wird gern aufgeräumt, und mit dem Ordner verschwinden auch die Daten.
4. Auf **Extrahieren** klicken und den Vorgang abwarten.

Nach dem Entpacken enthält der Ordner unter anderem `install.bat`, `start.bat`, den Ordner `app` und den Ordner `doku` mit den Anleitungen. Fehlt davon etwas, ist das Entpacken unvollständig geblieben.

Schritt 2 — Installieren

1. In den entpackten Ordner wechseln.
2. Doppelklick auf **`install.bat`**
3. Meldet Windows „Der Computer wurde durch Windows geschützt“: auf **Weitere Informationen** klicken, dann auf **Trotzdem ausführen**. Die Meldung erscheint bei allen Skripten ohne kostenpflichtige Signatur.
4. Das Skript prüft nun, ob Docker oder Python vorhanden ist.

Ist Python nicht installiert, bietet das Skript an, es einzurichten. Ein **J** genügt — die Installation läuft über den Windows-Paketverwalter und dauert einige Minuten. Danach muss das Fenster einmal geschlossen und `install.bat` erneut gestartet werden; Windows liest die Pfadangaben erst dann neu ein.

Ist Docker vorhanden, nutzt das Skript es. Die Anwendung läuft dann im Hintergrund und startet nach einem Neustart von selbst.

Schritt 3 — Verknüpfung und Autostart

Das Skript fragt zum Schluss zweierlei:

- **Verknüpfung auf dem Desktop** — bequemer als jedes Mal in den Ordner zu wechseln.
- **Beim Anmelden automatisch starten** — sinnvoll, wenn eine Wallbox über OCPP angebunden ist. Dann muss zusätzlich der Ruhezustand abgeschaltet werden: **Systemsteuerung** → **Energieoptionen**.

Täglicher Betrieb

Datei	Wofür
install.bat	Einmalig zur Einrichtung. Ein zweiter Aufruf schadet nicht.
start.bat	Startet die Anwendung. Das schwarze Fenster muss offen bleiben.

Ohne Docker muss das schwarze Fenster offen bleiben. Wird es geschlossen, ist die Anwendung nicht mehr erreichbar. Zum Beenden das Fenster schließen oder Strg + C drücken.

macOS und Linux

Auch hier gibt es ein Installationsskript. Es wird im Terminal aufgerufen.

1. Das Archiv entpacken — per Doppelklick oder im Terminal:

```
unzip eCharge_v12.10.zip -d ~/eCharge
```

2. Terminal öffnen und in den Ordner wechseln:

```
cd ~/eCharge/charge-at-home
```

3. Installation starten:

```
bash install.sh
```

4. Den Fragen folgen — Verzeichnis, Port, Zeitzone. Die Vorschläge in eckigen Klammern werden mit Enter übernommen.

Findet das Skript kein Docker, nennt es die passenden Befehle für das erkannte System. Ohne Docker lässt sich alternativ `./start.sh` verwenden — dann läuft die Anwendung direkt mit Python, solange das Terminalfenster offen ist.

Unter macOS meldet die Sicherheitsprüfung beim ersten Start möglicherweise einen unbekannten Entwickler. Über **Systemeinstellungen** → **Datenschutz & Sicherheit** lässt sich die Ausführung erlauben.

NAS, Raspberry Pi und Server

Der empfohlene Weg für den dauerhaften Betrieb. Das beiliegende Skript `install.sh` übernimmt die Einrichtung — es legt die Verzeichnisse an, erzeugt die Konfiguration, richtet auf Wunsch einen Zugangsschutz ein und startet die Anwendung.

Sie brauchen einen **SSH-Zugang** zu Ihrem Gerät. Unter Windows leistet das **PuTTY** oder die eingebaute PowerShell, unter macOS und Linux das Terminal.

Schritt 1 — Dateien auf das Gerät bringen

Variante A: über die Weboberfläche des NAS

1. Am NAS anmelden (DSM bei Synology, QTS bei QNAP).
2. In der **Dateiverwaltung** einen Ordner anlegen:
Synology: `docker/echarge-setup`
QNAP: `Container/echarge-setup`
3. Das entpackte Paket dorthin hochladen.

Variante B: unmittelbar über SSH

```
scp eCharge_v12.10.zip benutzer@192.168.1.50:/volume1/docker/  
ssh benutzer@192.168.1.50  
cd /volume1/docker && unzip eCharge_v12.10.zip
```

Schritt 2 — Installation starten

Per SSH anmelden, in das entpackte Verzeichnis wechseln und das Skript aufrufen:

```
cd /volume1/docker/charge-at-home  
bash install.sh
```

Das Skript fragt der Reihe nach:

Frage	Bedeutung
Verzeichnis	Wohin die Anwendung installiert wird. Dort landen Datenbank und Belege — dieses Verzeichnis gehört in die Datensicherung. Der Vorschlag passt meist.
Port	Unter welchem Port die Weboberfläche erreichbar ist. 8501 ist voreingestellt; ändern, falls belegt.
OCPP-Server	Nur nötig, wenn sich eine Wallbox unmittelbar hier anmelden soll. Für Loxone-Wallboxen und das LoxBerry-Plugin: nein.
Zeitzone	Bestimmt die Zeitstempel auf Belegen. Wird meist richtig erkannt.
Zugangsschutz	Siehe nächster Abschnitt.

Danach zeigt das Skript eine Zusammenfassung. Erst nach Bestätigung wird etwas verändert.

Zugangsschutz einrichten

Fragt das Skript nach dem Zugangsschutz und Sie antworten mit **J**, richtet es zusätzlich **Caddy** ein — einen vorgeschalteten Dienst, der Benutzername und Passwort abfragt, bevor die Anwendung überhaupt erreichbar wird.

1. **Benutzername** eingeben (Vorschlag: admin).
2. **Passwort** zweimal eingeben. Die Eingabe bleibt verborgen — das ist normal, es erscheinen keine Sternchen.
3. **Domain**: leer lassen für den Betrieb im Heimnetz. Wer einen Domainnamen besitzt, kann ihn eintragen — Caddy holt dann selbsttätig ein Zertifikat.

Das Passwort wird nicht im Klartext gespeichert. Das Skript lässt Caddy einen Hash erzeugen, der sich nicht zurückrechnen lässt.

Danach ist die Anwendung **nur noch über Caddy** erreichbar — der direkte Zugang ist geschlossen. Beim Aufruf im Browser erscheint eine Abfrage von Benutzername und Passwort.

Passwort ändern: Neuen Hash erzeugen mit

```
docker run --rm caddy:2-alpine caddy hash-password --plaintext 'neuesPasswort'
```

Die Ausgabe in die Datei Caddyfile eintragen, dann `docker compose restart caddy`.

Nach der Installation

Im Zielverzeichnis liegen vier Hilfsskripte für den täglichen Umgang:

Befehl	Wofür
<code>./start.sh</code>	Anwendung starten
<code>./stop.sh</code>	Anwendung anhalten
<code>./sichern.sh</code>	Datenbank und Belege in den Ordner <code>sicherungen/</code> packen
<code>./update.sh</code>	Nach einem Update neu bauen — sichert vorher automatisch

Ohne Skript — von Hand

Wer lieber selbst konfiguriert: Die Datei `docker-compose.yml` liegt bei und lässt sich unmittelbar verwenden.

```
cd charge-at-home
docker compose up -d
```

Für Portainer gibt es zusätzlich `portainer-stack.yml`. Da Portainer nicht aus einem lokalen Verzeichnis bauen kann, muss das Abbild vorher erzeugt werden:


```
docker build -t echarge/at-home:latest .  
docker save echarge/at-home:latest -o echarge.tar
```

Die Datei dann in Portainer unter **Images** → **Import** laden und den Stack anlegen.

Netzwerk

Port	Wofür	Wird benötigt
8501	Weboberfläche	immer
9000	OCPP 1.6-J	nur bei direkt angebundener Wallbox (Vollversion)

Port 9000 ist häufig belegt — Portainer nutzt ihn standardmäßig. Unter **Einstellungen** → **OCPP** lässt er sich ändern; die Anwendung prüft vorher, ob der neue Port frei ist. Die Adresse in der Wallbox muss dann angepasst werden.

Keine Freigabe ins Internet. Die Anwendung ist für den Betrieb im eigenen Netz ausgelegt. Eine Portweiterleitung im Router ist weder nötig noch empfohlen. Für den Zugriff von unterwegs eignet sich ein VPN.

Zugriff von unterwegs

Diese Anwendung hat **keine Benutzeranmeldung**. Wer die Adresse erreicht, sieht das vollständige Fahrtenbuch mit Adressen und Fahrzwecken, alle Ladevorgänge, die Zugangsdaten der Wallbox und die erzeugten Belege. Es gibt kein Passwort und keine Zugriffskontrolle.

Machen Sie die Anwendung nicht über das Internet erreichbar. Eine Portweiterleitung im Router genügt, damit sie binnen Stunden von automatischen Scannern gefunden wird. Für den Zugriff von unterwegs ist ein VPN der richtige Weg.

Der empfohlene Weg: VPN

Lösung	Wann sie passt
Router-VPN	Fritz!Box, Speedport und die meisten anderen bringen WireGuard mit. Einmal einrichten, dann ist das gesamte Heimnetz von überall erreichbar — nicht nur diese Anwendung.
NAS-VPN	Synology und QNAP haben einen VPN-Server als Paket. Gleichwertig zum Router-VPN.
Tailscale	Für Privatsnutzung kostenlos, läuft als Container neben der Anwendung. Keine Portfreigabe nötig, funktioniert auch hinter DS-Lite.

Zu Reverse Proxies

Ein vorgeschalteter Proxy wie **Caddy**, nginx oder Traefik stellt Verschlüsselung her — er ersetzt aber keine Zugangskontrolle. Ein gültiges Zertifikat bedeutet nur, dass die Verbindung nicht mitgelesen werden kann. Wer die Adresse aufruft, kommt trotzdem ungehindert hinein.

Wer dennoch einen Proxy einsetzt, sollte mindestens eine Zugangsabfrage davorschalten. Bei Caddy genügt dafür ein Eintrag im Caddyfile:

```
echarge.example.de {  
    basic_auth {  
        benutzer $2a$14$...  
    }  
    reverse_proxy 192.168.1.50:8501  
}
```

Den Passwort-Hash erzeugt caddy `hash-password`. Auch damit bleibt die Verantwortung für Absicherung und Datenschutz beim Betreiber.

Zugriff vom Handy oder Tablet

Im selben Netz genügt die Adresse des Geräts:

```
http://192.168.1.50:8501
```

Die eigene IP-Adresse nennt das Startskript beim Start. Bei Docker ist es die Adresse des NAS beziehungsweise des Raspberry Pi.

Daten sichern

Alle Daten liegen im Ordner `data/` — Datenbank, erzeugte Belege, Einstellungen. Diesen Ordner sichern, mehr ist nicht nötig.

Zusätzlich bringt die Anwendung eine eigene Sicherungsfunktion mit: **Einstellungen** → **System** → **Sicherung herunterladen**. Sie erzeugt ein ZIP-Archiv, das sich auf einem anderen Rechner wiederherstellen lässt.

Aktualisieren

Docker:

```
docker compose down  
docker compose up -d --build
```

Lokal: Neues Paket entpacken, den Ordner `data/` aus der alten Installation übernehmen, neu starten.

In beiden Fällen bleiben die Daten erhalten. Eine Sicherung vorab schadet trotzdem nicht.

Wenn etwas nicht funktioniert

Beim Start

Beobachtung	Ursache und Abhilfe
Seite nicht erreichbar	Läuft die Anwendung? Bei Docker: <code>docker ps</code> . Lokal: Ist das Fenster offen?
„Python wurde nicht gefunden“	Python installieren und dabei Add Python to PATH ankreuzen.
Port 8501 belegt	Ein anderes Programm nutzt ihn. In <code>docker-compose.yml</code> auf <code>8502:8501</code> ändern.
Docker: „permission denied“	Unter Linux: <code>sudo usermod -aG docker \$USER</code> , dann neu anmelden.
Fenster schließt sich sofort	Startskript aus dem Terminal aufrufen statt per Doppelklick — dann bleibt die Fehlermeldung stehen.

Im Betrieb

Beobachtung	Ursache und Abhilfe
Daten nach Update weg	Der Ordner <code>data/</code> wurde nicht eingebunden. Abschnitt <code>volumes</code> : prüfen.
Zeitstempel falsch	TZ in der Compose-Datei anpassen, etwa <code>Europe/Berlin</code> .
Ladungen fehlen	Lief die Anwendung zum Zeitpunkt der Ladung? Bei lokaler Installation die häufigste Ursache.
Vom Handy nicht erreichbar	Gleiches Netz? Firewall des Rechners prüfen — Port 8501 muss erreichbar sein.

Nach der Installation

Beim ersten Start führt die Anwendung durch vier Fragen: Hinweis bestätigen, Name, Abrechnungsfall und Strompreis. Danach fragt sie, ob Beispieldaten angelegt werden sollen — damit lassen sich alle Funktionen ausprobieren, ohne selbst etwas zu erfassen.

Wie es weitergeht, beschreibt der **Schnellstart**. Das **Benutzerhandbuch** erklärt jede Funktion im Einzelnen. Beide liegen dem Paket im Ordner `doku/` bei.

Bei Fragen: www.loewemann.com